

Analiza potrebe za uvođenjem paterna ponašanja u sistem BentoLab

Uvod

BentoLab predstavlja sistem za naručivanje personalizovanih bento torti koji korisnicima omogućava kreiranje i praćenje narudžbi, odabir karakteristika torte, procjenu složenosti izrade, pregled statusa narudžbe te primanje obavještenja o njenom toku. Tokom razvoja sistema uočeno je da određeni dijelovi implementacije mogu postati teško održivi ukoliko se u budućnosti bude povećavao broj funkcionalnosti i poslovnih pravila.

Kako bi se sistem lakše proširivao i održavao, potrebno je primijeniti odgovarajuće obrasce dizajna iz grupe paterna ponašanja. Ovi paterni omogućavaju jasnije definisanje komunikacije između objekata i jednostavniju organizaciju poslovne logike.

U nastavku je predstavljena analiza paterna ponašanja koji se prirodno uklapaju u postojeću arhitekturu sistema: Strategy, Observer, State, Command, Template Method, Chain of Responsibility, Mediator i Memento.

Strategy Pattern

Jedna od ključnih funkcionalnosti sistema BentoLab jeste procjena složenosti i cijene personalizovane torte. Konačna cijena ne zavisi samo od osnovnih karakteristika proizvoda, već i od dodatnih zahtjeva korisnika kao što su tekst na torti, referentna slika, složenost dekoracije ili specifične prilagodbe.

Trenutna implementacija obračuna cijene mogla bi se realizovati velikim brojem uslovnih naredbi koje bi vremenom postajale sve složenije. Svako dodavanje nove opcije zahtijevalo bi izmjene postojećeg koda, što povećava mogućnost nastanka grešaka.

Primjenom Strategy paterna svaki način obračuna cijene može se izdvojiti u zasebnu strategiju. Na taj način sistem može birati odgovarajući algoritam obračuna u zavisnosti od karakteristika narudžbe, bez potrebe za izmjenom postojećih klasa.

Prednost ovakvog pristupa ogleda se u jednostavnijem održavanju sistema, lakšem testiranju pojedinačnih algoritama i mogućnosti dodavanja novih pravila obračuna bez uticaja na ostatak aplikacije.

Observer Pattern

U okviru sistema BentoLab korisnici očekuju pravovremene informacije o statusu svojih narudžbi. Tokom procesa izrade torta prolazi kroz više faza, pri čemu svaka promjena statusa predstavlja važnu informaciju za korisnika.

Primjeri takvih promjena su:

- narudžba uspješno kreirana,
- narudžba potvrđena,
- torta je u procesu izrade,
- torta je završena,
- narudžba je spremna za preuzimanje ili dostavu.

Direktno povezivanje modula za upravljanje narudžbama sa modulom za obavještenja dovelo bi do povećane zavisnosti između komponenti sistema.

Observer pattern omogućava da objekat narudžbe automatski obavještava zainteresovane komponente kada dođe do promjene statusa. Na taj način logika obavještavanja postaje nezavisna od logike upravljanja narudžbom, što rezultira fleksibilnijim i preglednijim rješenjem.

Dodatna prednost ovog pristupa jeste mogućnost jednostavnog proširenja sistema novim tipovima obavještenja bez izmjena u postojećoj implementaciji.

State Pattern

Narudžba u sistemu BentoLab prolazi kroz više međusobno povezanih stanja. Svako stanje definiše skup dozvoljenih aktivnosti i mogućih prelaza u naredna stanja.

Tipičan životni ciklus narudžbe može obuhvatati sljedeća stanja:

- Kreirana
- Potvrđena
- U izradi
- Spremna za preuzimanje
- Preuzeta

Kako broj stanja raste, implementacija zasnovana na velikom broju uslovnih naredbi postaje nepregledna i teška za održavanje.

State pattern omogućava da se ponašanje sistema organizuje kroz zasebne klase koje predstavljaju pojedina stanja narudžbe. Na taj način svako stanje preuzima odgovornost za vlastita pravila i dozvoljene akcije.

Primjena ovog paterna pojednostavljuje upravljanje životnim ciklusom narudžbe, smanjuje kompleksnost poslovne logike i olakšava uvođenje novih stanja u budućim verzijama sistema.

Command Pattern

U sistemu BentoLab svakodnevno se izvršava veliki broj korisničkih i administrativnih akcija. Kreiranje narudžbe, potvrđivanje zahtjeva, otkazivanje narudžbe, promjena statusa izrade i evidentiranje završetka procesa predstavljaju zasebne operacije koje sistem mora obraditi.

Ukoliko bi sve ove aktivnosti bile implementirane direktno kroz kontrolere ili korisnički interfejs, došlo bi do snažne povezanosti između slojeva aplikacije. Takvo rješenje postalo bi teško za održavanje kako bi broj funkcionalnosti rastao.

Command pattern omogućava da se svaka akcija predstavi kao poseban objekat koji sadrži sve informacije potrebne za njeno izvršavanje. Na primjer, kreiranje narudžbe, potvrđivanje narudžbe i otkazivanje narudžbe mogu biti predstavljeni zasebnim komandama koje sistem izvršava nezavisno od načina pokretanja zahtjeva.

Prednost ovog pristupa ogleda se u boljoj organizaciji poslovne logike, jednostavnijem evidentiranju aktivnosti korisnika i mogućnosti proširenja sistema novim operacijama bez izmjena postojećeg koda.

Template Method Pattern

Proces kreiranja narudžbe u sistemu BentoLab sastoji se od više koraka koji se uvijek izvršavaju istim redoslijedom. Korisnik najprije bira osnovne karakteristike torte, zatim dodaje dekoracije i dodatne zahtjeve, nakon čega sistem vrši procjenu cijene i evidentira narudžbu.

Iako osnovna struktura procesa ostaje ista, pojedini koraci mogu se razlikovati zavisno od vrste narudžbe. Na primjer, narudžbe koje sadrže referentnu sliku mogu zahtijevati dodatnu provjeru, dok jednostavnije narudžbe mogu preskočiti određene korake.

Template Method pattern omogućava definisanje zajedničke strukture procesa u baznoj klasi, dok se pojedini koraci prilagođavaju kroz izvedene klase.

Primjena ovog paterna doprinosi dosljednosti poslovnih procesa, smanjuje dupliranje koda i omogućava jednostavnije proširenje sistema novim tipovima narudžbi.

Chain of Responsibility Pattern

Prije prihvatanja narudžbe sistem mora izvršiti više provjera kako bi osigurao ispravnost unesenih podataka. Potrebno je provjeriti da li su uneseni svi obavezni podaci, da li je odabran validan datum, da li postoji slobodan termin za izradu torte te da li su svi podaci uneseni u odgovarajućem formatu.

Implementacija svih provjera unutar jedne metode dovela bi do složenog i teško održivog rješenja.

Chain of Responsibility pattern omogućava da svaka provjera bude predstavljena zasebnim objektom odgovornim za određeni dio validacije. Nakon izvršavanja jedne provjere zahtjev se automatski proslijeđuje narednoj komponenti u lancu.

Na taj način sistem postaje fleksibilniji i jednostavniji za proširenje. Dodavanje novih pravila validacije ne zahtijeva izmjene postojećih provjera, već samo uključivanje nove karike u postojeći lanac odgovornosti.

Mediator Pattern

Tokom rada sistema BentoLab više modula međusobno sarađuje. Modul za narudžbe komunicira sa modulom za korisnike, modulom za procjenu cijene, modulom za dostavu i modulom za obavještenja.

Direktna komunikacija između svih komponenti dovela bi do velikog broja međusobnih zavisnosti koje bi vremenom otežale održavanje sistema.

Mediator pattern uvodi centralnu komponentu koja upravlja komunikacijom između različitih dijelova aplikacije. Umjesto direktnog povezivanja, svi moduli razmjenjuju informacije preko posrednika.

Primjena ovog paterna smanjuje povezanost između komponenti, pojednostavljuje arhitekturu sistema i omogućava lakše dodavanje novih funkcionalnosti bez izmjena postojećih komunikacionih veza.

Memento Pattern

Korisnici sistema BentoLab tokom procesa personalizacije torte često prolaze kroz više koraka i mijenjaju prethodno odabrane opcije. Tokom uređivanja narudžbe može se javiti potreba za vraćanjem na raniju verziju konfiguracije.

Bez posebnog mehanizma za čuvanje prethodnih stanja sistem bi morao ponovo unositi sve podatke ili održavati složene strukture za evidentiranje promjena.

Memento pattern omogućava čuvanje prethodnih stanja objekta bez narušavanja njegove unutrašnje strukture. U kontekstu sistema BentoLab ovaj pristup može se koristiti za privremeno spremanje konfiguracije torte tokom procesa kreiranja narudžbe.

Primjena ovog paterna unapređuje korisničko iskustvo, omogućava jednostavnije vraćanje na prethodne korake i smanjuje vjerovatnoću gubitka podataka tokom procesa naručivanja.

Zaključak

Analizom postojećih funkcionalnosti sistema BentoLab zaključeno je da primjena paterna ponašanja može značajno unaprijediti njegovu arhitekturu. Strategy pattern omogućava fleksibilnu organizaciju logike za obračun cijene i procjenu složenosti torti. Observer pattern unapređuje mehanizam obavještanja korisnika i smanjuje povezanost između komponenti sistema. State pattern pojednostavljuje upravljanje statusima narudžbi i omogućava jasnije modelovanje poslovnih procesa.

Pored navedenih rješenja, Command pattern unapređuje organizaciju korisničkih i administrativnih akcija, Template Method standardizuje poslovne procese, Chain of Responsibility pojednostavljuje validaciju podataka, Mediator smanjuje međusobne zavisnosti između modula, dok Memento omogućava čuvanje prethodnih stanja i unapređuje korisničko iskustvo.

Uvođenjem navedenih paterna sistem postaje lakši za održavanje, jednostavniji za proširenje i spremniji za implementaciju novih funkcionalnosti u budućnosti.